



# KubePix: A Kubernetes-Based Image Sharing and Processing Pipeline

West Chester University  
Maiti Clark  
CSC581- Cloud Computing

# Chapter 1: Project Vision

## Project Overview

KubePix is an image-sharing web application designed to provide users with a simple way to upload and share photos through an online gallery. When a user uploads an image, KubePix automatically processes the image by resizing and compressing it to reduce its file size while maintaining suitable visual quality. The optimized image is then stored and made available for display within the application's photo feed.

The goal of KubePix is to combine a familiar photo-sharing experience with an automated image-processing pipeline. Rather than requiring users to manually resize or optimize their photos before uploading them, KubePix handles this process automatically. This creates a simple experience for the user while demonstrating how multiple application components can work together to receive, process, store, and deliver image content.

Behind the application, KubePix will use a Kubernetes-based architecture to separate these responsibilities across multiple components. This design will allow the project to demonstrate containerized application deployment, communication between services, persistent storage, and scalable image processing while presenting the user with a straightforward web interface.

## Motivation and Purpose

Modern photo-sharing applications allow users to upload images directly from their phones, cameras, and other devices. These images can vary significantly in file size and resolution, and high-resolution photographs may be much larger than necessary for displaying on a website. Storing and delivering unnecessarily large image files can consume additional storage space, increase network usage, and result in slower loading times for users.

KubePix is designed to simplify this process by automatically optimizing images after they are uploaded. Instead of requiring users to resize or compress their photos before sharing them, the application will handle these tasks as part of its image-processing pipeline. Users will be able to upload an image in its original form while KubePix prepares an optimized version for storage and display on the website.

In addition to providing a convenient experience for users, KubePix provides an opportunity to explore how a web application can be divided into independent services within a cloud environment. The project will use Kubernetes to coordinate the different stages of the application, allowing components responsible for receiving uploads, processing images, storing data, and serving content to communicate while remaining separated. This architecture demonstrates how containerized services can work together as a complete application and provides a foundation that could be expanded or scaled as demand increases.

## User Stories and Workflow

The KubePix workflow is designed to make uploading and sharing images simple for the user while handling image optimization automatically in the background. To define the expected functionality of the application, the following user stories describe the primary interactions a user will have with KubePix.

### User Story 1 – Upload an Image:

As a user, I want to upload an image to KubePix so that I can share it through the online photo feed. After an image is uploaded, the system will automatically send it through the image-processing pipeline.

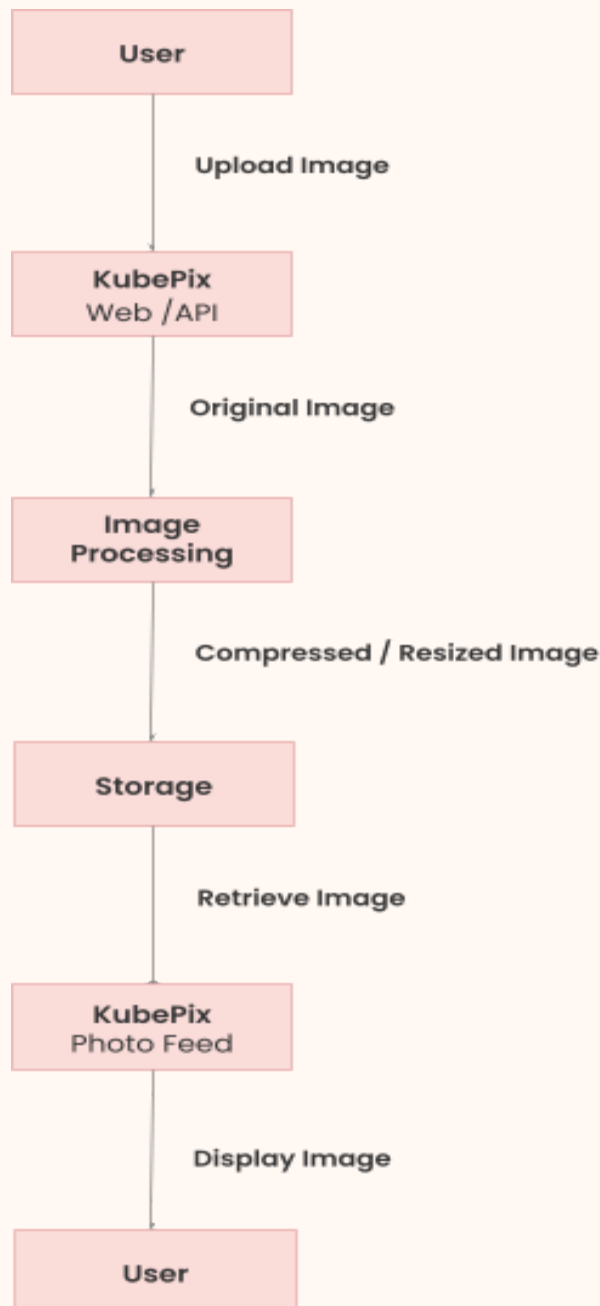
### User Story 2 – Automatic Image Processing:

As a user, I want my uploaded image to be automatically resized and compressed so that I do not have to manually optimize the image before sharing it. KubePix will process the uploaded image and prepare an optimized version for web display.

### User Story 3 – View Uploaded Images:

As a user, I want to view my processed images in the KubePix photo feed so that I can easily access and view the photos that have been uploaded to the application.

### Workflow



At a high level, a user's interaction with KubePix begins when an image is selected and uploaded through the website. The application receives the image and passes it to the image-processing stage, where the image is resized and compressed. The processed image and its associated metadata are then stored by the application. Once processing is complete, the optimized image becomes available for retrieval and display in the KubePix photo feed.

Figure 1. KubePix Image Processing Workflow

## High-Level Architecture

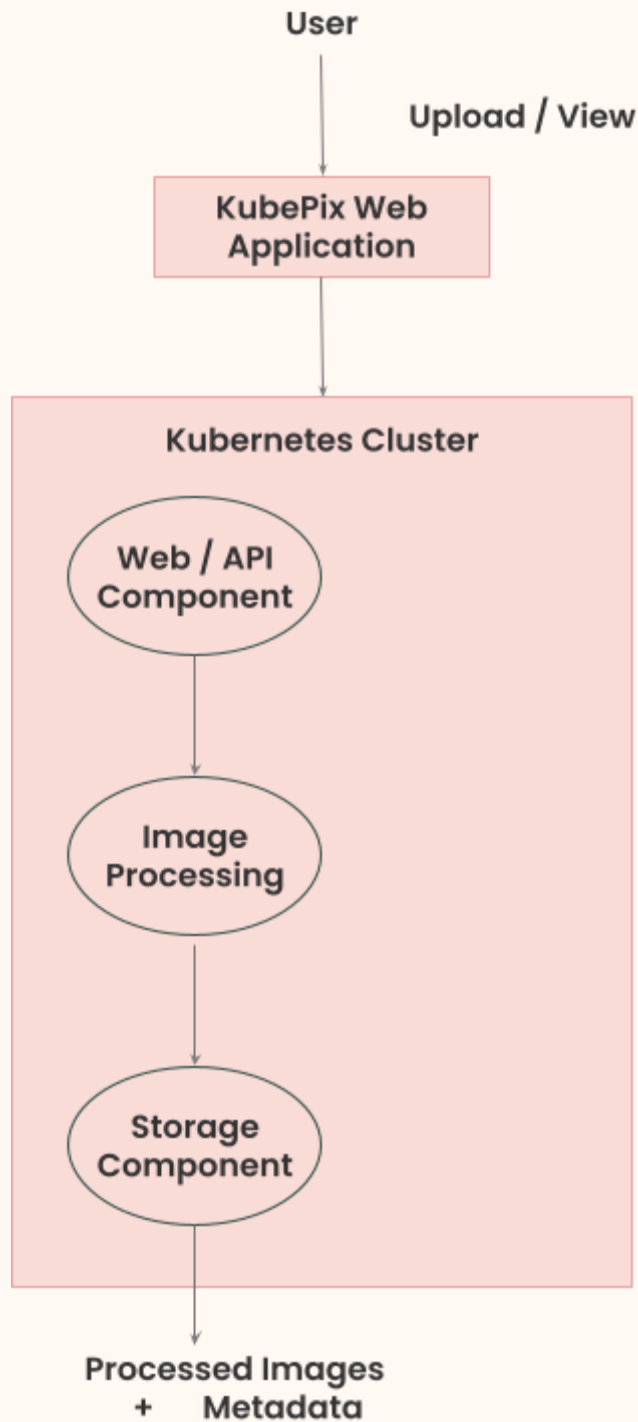


Figure 2. High-Level KubePix System Architecture

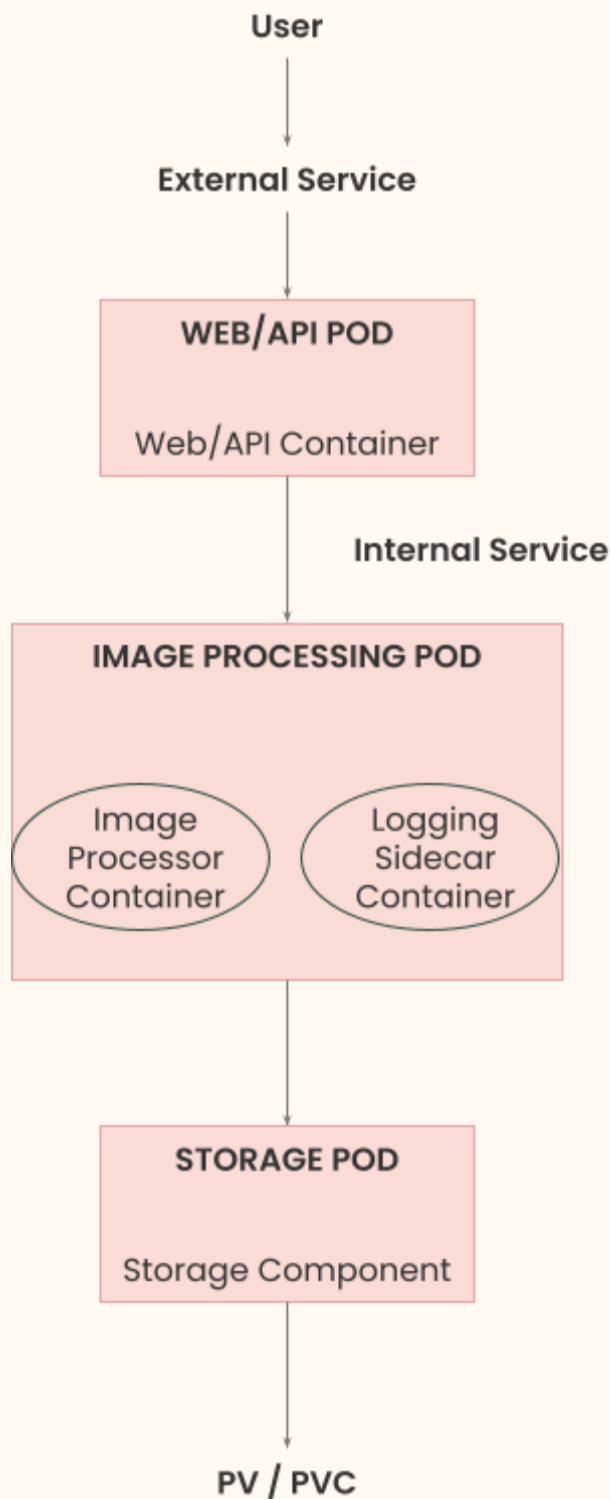
KubePix is designed using a multi-stage architecture in which separate components are responsible for receiving, processing, storing, and displaying images. The application will be deployed within a Kubernetes environment, allowing these components to operate independently while communicating as part of a single image-sharing pipeline. Figure 2 presents a high-level view of the proposed KubePix architecture.

The KubePix architecture separates the application's primary responsibilities into multiple components within a Kubernetes cluster. The Web/API component acts as the main entry point for the application and handles requests from users, including image uploads and requests to retrieve previously processed images. Uploaded images are passed to the image-processing component, which is responsible for resizing and compressing each image into a format suitable for web display.

After processing is complete, the optimized image and its associated metadata are sent to the storage component, where they can be persisted and later retrieved by the application. When a user accesses the KubePix photo feed, the application retrieves the appropriate stored content and displays the processed images. Separating these responsibilities allows individual portions of the application to be managed independently and provides a foundation for scaling components that may experience heavier workloads, particularly the image-processing stage.

# Chapter 2: Implementation Plan

## Kubernetes Architecture



KubePix will be deployed as a multi-Pod application within a Kubernetes cluster. The architecture will separate the primary responsibilities of the application into Web/API, image-processing, and storage components. Separating these functions allows each component to be deployed and managed independently while still participating in the overall KubePix image-processing pipeline.

The Web/API Pod will serve as the primary entry point for application requests. It will receive image uploads from users and coordinate communication with the internal components of the application. It will also handle requests to retrieve processed images for display within the KubePix photo feed.

The Image Processing Pod will be responsible for resizing and compressing uploaded images. This Pod will use a multi-container design consisting of a primary image-processing container and a logging sidecar container. The image-processing container will perform the actual image optimization, while the sidecar container will collect processing logs or related information. Using these containers within the same Pod allows them to operate closely together while maintaining separate responsibilities.

The Storage Pod will be responsible for storing and retrieving processed images and their associated metadata. Persistent Kubernetes storage will be used so that stored content is not lost if an application Pod is restarted or replaced. The specific PersistentVolume and PersistentVolumeClaim configuration will be defined as part of the application's storage implementation.

Figure 3. Proposed KubePix Kubernetes Architecture

## Service and Networking Design

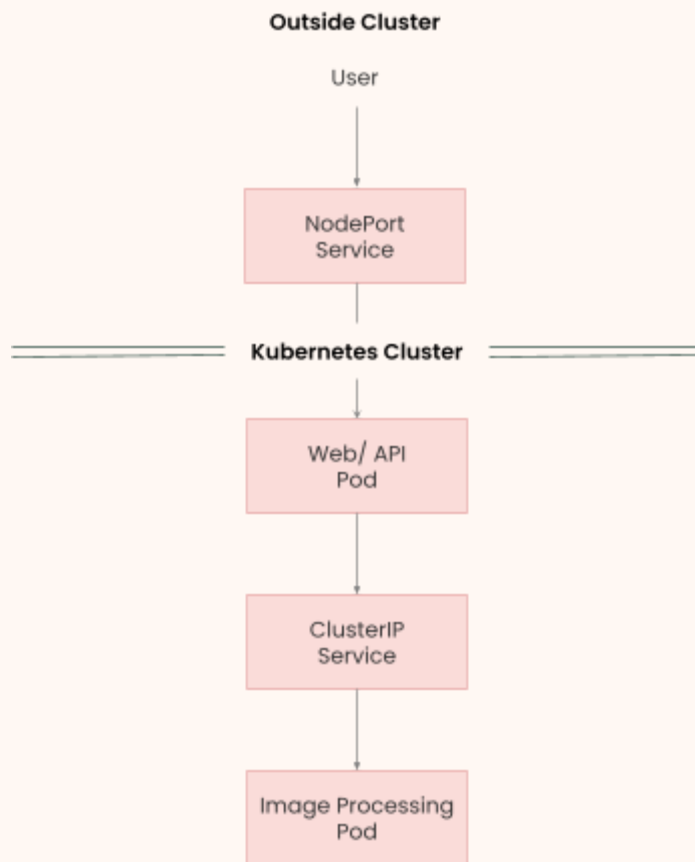


Figure 4. KubePix Service and Networking Design

KubePix will use Kubernetes Services to provide communication between the different components of the application. The Web/API component will require external access because it serves as the primary point of interaction between users and the application. A NodePort Service will be used to expose the Web/API component outside of the Kubernetes cluster, allowing users to access the KubePix web application and submit image uploads.

Communication between internal application components will use ClusterIP Services. For example, the Web/API component will communicate with the image-processing component through an internal ClusterIP Service. This allows the image-processing component to receive requests from the Web/API without exposing it directly outside the Kubernetes cluster. Internal services provide stable network access to the appropriate Pods even if individual Pods are restarted or replaced.

This design separates external and internal network traffic. Only the components that require direct user access will be exposed outside of the cluster, while image processing and storage functionality will remain accessible only to the application components that require them.

## API Design

KubePix will provide a REST-based API that allows the web application to interact with the image-processing and storage components of the system. The initial API will focus on the primary functionality of the application: uploading images and retrieving processed images for display. Keeping the API limited to these core operations will provide the functionality necessary for the initial version of KubePix while allowing additional features to be added in the future.

When a user uploads an image, the Web/API component will receive the request through the POST `/images` endpoint. The uploaded image will then be passed to the image-processing component for resizing and compression before being stored. The GET `/images` endpoint will retrieve available processed images for the KubePix photo feed, while GET `/images/{id}` will allow the application to retrieve information about a specific uploaded image.

| Method | Endpoint                  | Description                                   |
|--------|---------------------------|-----------------------------------------------|
| POST   | <code>/images</code>      | Uploads a new image to KubePix for processing |
| GET    | <code>/images</code>      | Returns a collection of processed images      |
| GET    | <code>/images/{id}</code> | Returns information for a specific image      |

### Example upload response:

After someone successfully uploads dog.jpg, KubePix should respond with:

```
{
  "id": "1024",
  "filename": "dog.jpg",
  "status": "processed",
  "imageUrl": "/images/1024"
}
```

The response provides a unique image identifier, the original filename, the current processing status, and the location that can be used to retrieve the processed image.

## Persistence and Storage Plan

KubePix will require persistent storage to ensure that processed images and their associated metadata remain available even if application Pods are restarted or replaced. Because Kubernetes Pods are temporary by nature, storing uploaded content only within a container's local filesystem could result in the loss of data when a Pod is recreated.

The application will use a Kubernetes PersistentVolume (PV) and PersistentVolumeClaim (PVC) to provide persistent storage for the storage component. The PersistentVolume will provide the underlying storage space, while the PersistentVolumeClaim will allow the application to request and access that storage. The storage component will mount the persistent volume so that processed images remain available independently of the lifecycle of an individual Pod.

KubePix will store the optimized image files along with metadata associated with each upload. This metadata may include a unique image identifier, the original filename, processing status, and the location of the processed image. Separating the persistent data from the application containers will allow KubePix to maintain uploaded content while individual application components are updated, restarted, or replaced.

```
/kubepix-data
├── images/
│   ├── 1024-dog.jpg
│   ├── 1025-cat.jpg
│   └── ...
└── metadata/
    ├── 1024.json
    ├── 1025.json
    └── ...
```

## Scaling Plan

KubePix will be designed so that the image-processing component can be scaled as application demand increases. Image resizing and compression are expected to be among the most resource-intensive operations in the application, particularly when multiple users upload images at the same time. Increasing the number of image-processing Pods would allow multiple processing requests to be handled without relying on a single Pod.

The image-processing component will be managed using a Kubernetes Deployment, allowing multiple replicas of the Pod to run when additional processing capacity is required. Requests can then be distributed among the available image-processing Pods through the component's ClusterIP Service. This approach allows processing capacity to be increased without requiring the entire KubePix application to be duplicated.

The initial version of KubePix may operate with a small number of replicas while the application is being

developed and tested. The number of replicas can later be increased as workload requirements are evaluated.

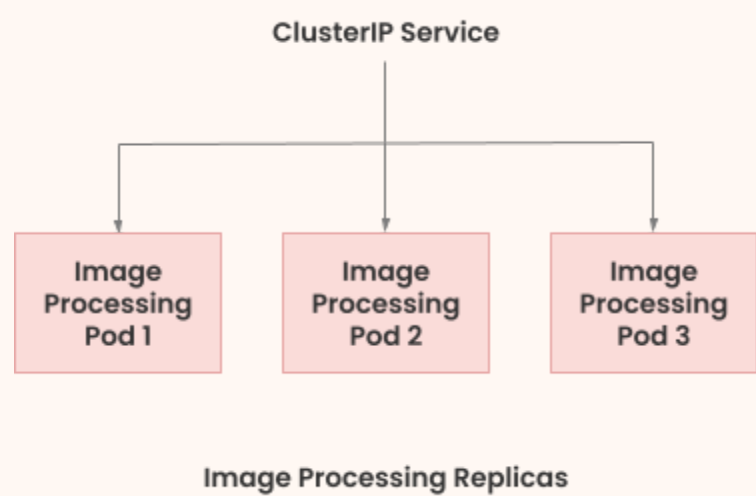


Figure 5. Proposed Scaling of the KubePix Image Processing Component

## Risk and Test Plan

Testing will be important to ensure that KubePix can reliably accept, process, store, and display images under different conditions. Because the application consists of multiple components communicating within Kubernetes, failures may occur during image uploads, processing, storage, or communication between Pods. Testing will therefore focus on both normal user interactions and situations that could cause the image-processing pipeline to fail.

| Risk                     | Potential Impact                                           |
|--------------------------|------------------------------------------------------------|
| Large image files        | May require excessive processing time or resources         |
| Invalid file uploads     | Non-image or corrupted files could cause processing errors |
| Pod or container failure | Could interrupt part of the image-processing pipeline      |
| Storage failure          | Processed images or metadata may become unavailable        |

These risks will be addressed through input validation, persistent storage, Kubernetes Pod management, and application-level error handling. Kubernetes will also allow failed application Pods to be restarted or replaced, while persistent storage will help ensure that stored images remain available independently of individual Pod lifecycles.

### Test Matrix:

| Test Case           | Input/Condition             | Expected Result                                     |
|---------------------|-----------------------------|-----------------------------------------------------|
| Normal image upload | Valid .jpg image            | Image is uploaded, processed, stored, and displayed |
| Large image upload  | High-resolution/large image | Image is successfully resized and compressed        |



|                               |                                    |                                                                                   |
|-------------------------------|------------------------------------|-----------------------------------------------------------------------------------|
| <b>Invalid file upload</b>    | Non-image file such as .txt        | Upload is rejected with an appropriate error                                      |
| <b>Corrupted image</b>        | Invalid or damaged image file      | Processing fails safely and an error is returned                                  |
| <b>Pod failure</b>            | Image-processing Pod is terminated | Kubernetes replaces the failed Pod and processing service becomes available again |
| <b>Stored image retrieval</b> | Request an existing image ID       | Correct processed image is returned                                               |
| <b>Invalid image ID</b>       | Request an ID that does not exist  | Application returns an appropriate not-found response                             |

Successful testing will require KubePix to complete the full upload-to-display workflow for valid images while handling invalid input and component failures without causing the overall application to become unavailable.

## Team Charter

KubePix will be developed as an individual project. As the sole developer, I will be responsible for the design, development, deployment, testing, and documentation of the application. This includes developing the web interface and API, implementing the image-processing pipeline, configuring the Kubernetes resources, managing persistent storage, and testing the completed system.

| Category           | Plan                                                                                                                                                        |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Project Role       | Sole developer responsible for application development, Kubernetes configuration, testing, and documentation                                                |
| Work Cadence       | Project progress will be reviewed weekly, with development tasks organized according to course milestones and deliverable deadlines                         |
| Definition of Done | A task is considered complete when it has been implemented, tested successfully, documented where necessary, and committed to the project GitHub repository |

The project will be developed incrementally so that each course milestone builds upon the previous stage. This approach will allow the architecture and implementation to be evaluated throughout development rather than waiting until the completion of the entire application.

# MAITI F. CLARK

484-889-8639 | [maiticlark@gmail.com](mailto:maiticlark@gmail.com) | [LinkedIn](#) | West Chester, PA

## EDUCATION

---

West Chester University | West Chester, PA

January 2025 -

Current

*M.S.- Masters of Computer Science*

- **Courses:** Programming Languages, Advanced Seminar: Computer Security, Foundations of Computer Science, Networks and Data Communications, Advanced Seminar: Data Visualization
- **Upsilon Pi Epsilon (UPE) Honor Society Member** – West Chester University Chapter

University College Dublin | Dublin, Ireland

September 2018 -

May 2021

*B.A. - Double Major in Information & Social Computing with Sociology*

- **Courses:** Artificial Intelligence, The Digital Self, Information Ethics, Digital Marketing, Web Publishing, Information Architecture: Designing the Web, Consumer & Buyer Behavior, Information & Collaboration in Organisations, Computer-Mediated Communication, Social Media & Participation in an Online World

## PROFESSIONAL EXPERIENCE

---

La Scala's Fire, Newtown Square PA | Lead Bartender

November 2024 -

Present

- Serve and engage 50+ guests per shift while cultivating lasting customer relationships and ensuring high-quality service.
- Train new hires on menu knowledge, POS systems, and company standards while supporting management in maintaining efficient, consistent bar operations.

Irish Dance Teacher, Folsom PA | Instructor

January 2014 -

Present

- Lead classes for students of all ages and skill levels, fostering growth, confidence, and engagement through adaptable teaching methods.
- Collaborate with fellow instructors to design curriculum and maintain a cohesive, supportive learning environment.

Unifeyed LLC, West Chester PA | Web Specialist

November 2021 - November

2024

- Directed full-cycle web design and development, ensuring scalability, usability, and secure coding practices.
- Led client workshops to translate requirements into secure, mobile-responsive prototypes and applications.
- Applied authentication, access control, and secure data handling to maintain confidentiality, integrity, and availability.
- Utilized Jira and Teamwork to coordinate Agile workflows and track development pipelines.
- Managed digital content and client social media accounts, incorporating monitoring, analytics, and risk awareness.

Sidelines Sportswear, Exton PA | Marketing & Operations Intern

July 2021 - October

2021

- Directed social media growth initiatives through targeted campaigns and digital content creation to showcase new product offerings.
- Expanded company reach by engaging clients across multiple industry segments.

## SKILLS

---

**Languages (by proficiency):** Java, Python, OCaml, Assembly, HTML, CSS

**Tools & Frameworks:** Git, Cisco Packet Tracer, Linux, Microsoft Office

**Security & Networking:** Firewalls, NAT, Routing & Switching (via coursework/labs), Network Protocols

**Other:** Figma, InVision, Canva

## PROJECTS

---

**Network Security Lab** – Configured firewalls, NAT, and routing rules in Cisco Packet Tracer to enable secure cross-domain communication.

**Programming Languages Project** – Built a parser and lexical analyzer in Java to handle custom inputs and validate syntax.

**Data Security Lab** – Used OllyDbg to analyze and bypass a nag screen, demonstrating reverse engineering fundamentals.